

# One Tool Is Enough: Reinforcement Learning of LLM Agents for Repository-Level Code Navigation

Zhaoxi Zhang<sup>1 2 \*</sup> Yitong Duan<sup>3</sup> Yanzhi Zhang<sup>3</sup> Yiming Xu<sup>2</sup> Zhixiang Wang<sup>1 2</sup> Kun Liang<sup>1 2</sup> Weikang Li<sup>4</sup>  
Jiahui Liang<sup>4</sup> Deguo Xia<sup>4</sup> Jizhou Huang<sup>4</sup> Jiyao He<sup>3</sup> Shuxin Zheng<sup>3</sup> Yunfang Wu<sup>1 2</sup>

## Abstract

Locating files and functions requiring modification in large software repositories is challenging due to their scale and structural complexity. Existing LLM-based methods typically treat this as a repository-level retrieval task and rely on multiple auxiliary tools, which often overlook code execution logic and complicate model control. We propose **RepoNavigator**, an LLM agent equipped with a **single execution-aware tool**: jumping to the definition of an invoked symbol. This unified design reflects the actual flow of code execution while simplifying tool manipulation. RepoNavigator is **trained end-to-end via Reinforcement Learning (RL)** directly from a base pretrained model, without relying on closed-source distillation. Experiments demonstrate that RL-trained RepoNavigator achieves state-of-the-art performance, with the 7B model outperforming 14B baselines, the 14B model surpassing 32B competitors, and the 32B model exceeding closed-source models such as GPT-5 on most metrics. These results confirm that integrating a **single, structurally grounded tool with RL training** provides an efficient and scalable solution for repository-level issue localization.

## 1. Introduction

With the rapid advancement of Large Language Models (LLMs) (Liu et al., 2024a; Team, 2024; Yang et al., 2025a), equipping LLMs with pre-built tools to form LLM agents

\*Work done during internship at ZGCI. <sup>1</sup>National Key Laboratory for Multimedia Information Processing, Peking University <sup>2</sup>School of Computer Science, Peking University <sup>3</sup>Zhongguancun Institute of Artificial Intelligence (ZGCI) <sup>4</sup>Baidu Inc. Correspondence to: Yitong Duan <duanyitong@zgci.ac.cn>, Yunfang Wu <wuyf@pku.edu.cn>.

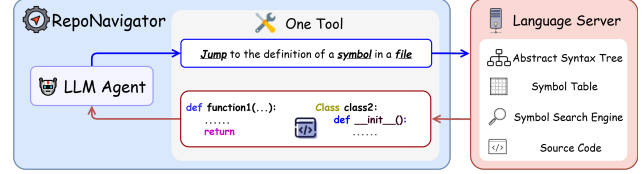


Figure 1. Illustration of an LLM navigating through a code repository. The LLM is equipped with a single yet powerful tool: `jump`, implemented via a language server.

has become a common paradigm for expanding their capabilities (Shen, 2024; Yuan et al., 2024; Lu et al., 2024). In the domain of software engineering (SWE), although LLM agents can effectively handle simple programming tasks (Hui et al., 2024; Guo et al., 2024a), their ability to operate on large-scale software repositories remains limited. SWE-bench (Jimenez et al., 2023) currently serves as the most comprehensive benchmark for evaluating whether LLMs can resolve real-world GitHub issues. All pretrained LLMs cannot process the whole repository directly due to context limits (Yang et al., 2024b). While SWE-Agent (Jimenez et al., 2023) provides moderate gains, it remains far from enabling robust repository-level reasoning.

Most existing agents rely on test-time scaling applied directly to pretrained LLMs (Liu et al., 2024c; Chen et al., 2025c; Schmidgall et al., 2025). In SWE tasks, tools are essential rather than optional: real-world repositories are far larger than the context window of current LLMs, making it impossible to process an entire codebase in a single forward pass. Agents must therefore iteratively invoke tools to retrieve partial information from the repository and interleave natural-language reasoning with tool calls.

Mainstream LLM agents (Chen et al., 2025c; Liu et al., 2025b; Xiang et al., 2025; Wang et al., 2023; Chen et al., 2024) are rarely exposed to such agentic interaction patterns during pretraining and typically acquire tool usage only through few-shot prompting, which is insufficient for learning complex multi-step tool-chaining behaviors. Moreover, because tool definition spaces are effectively unbounded, pretrained models cannot fully internalize their semantics without post-training. To mitigate these issues, post-training

paradigms such as Supervised Finetuning (SFT) (Ma et al., 2025) and Reinforcement Learning with Verifiable Rewards (RLVR) (Yu et al., 2025a; Yue et al., 2025) have been applied, with promising results in domains including retrieval agents (Jin et al., 2025a), and GUI agents (Hong et al., 2024)

Directly training an agent to fix software issues, however, remains difficult. A single bug often admits multiple valid patches, making string-level evaluation unreliable. The only precise evaluation method requires executing candidate patches inside a dedicated Docker environment for each repository (Luo et al., 2025), which is prohibitively expensive in terms of CPU resources for supervised training. To make training more tractable, we adopt a simplified yet widely generalizable assignment: **issue localization**. Prior work shows that a software issue becomes substantially easier to resolve once the relevant functions and files are correctly identified (Chen et al., 2025c; Ma et al., 2025; Xia et al., 2024; Jiang et al., 2025). Since modern software repositories contain a significant amount of code—far beyond any LLM’s context window—localization drastically reduces the search space and improves downstream solvability. Crucially, localization outputs a discrete set of paths, enabling verifiable, string-level evaluation that is compatible with scalable training frameworks such as SFT and RLVR (string-level evaluation requires little resource).

Existing localization agents (Ma et al., 2025; Chen et al., 2025c; He et al., 2025) typically rely on multiple tools, including `SearchClass`, `SearchMethods`, and `GetImports`. Although effective to some extent, these tools consider high-level abstractions (classes, functions, etc) of programming languages, which do not reflect how code actually executes. High-level abstractions, such as classes or inheritance, disappear after compilation, leaving only sequential execution and `jump` operations. Since modern LLMs already excel at capturing sequential dependencies, we focus on enhancing their ability to `jump` across the repository—that is, to follow and inspect the source definition of symbols as they appear in execution. To this end, we introduce a single, structurally grounded tool: `jump`, which retrieves the precise definition of a given symbol. Our agent invokes this single tool to navigate in the repository, as depicted in Fig. 1. Details of this tool are provided in Sec. 3.3.

Our main contributions are threefold: (1) We propose the first repo-level localization agent trained using reinforcement learning directly from the pretrained model, without relying on distillation from a closed-source model. (2) We design a repository navigation agent that operates by performing realistic `jump` operations aligned with actual execution semantics. (3) We demonstrate that one unified tool significantly improves efficiency and controllability compared to multi-tool pipelines.

## 2. Related Works

### 2.1. Agentic Training

LLM agents are promising methods to equip models with complex tools while reasoning (Li et al., 2024; Huang et al., 2024; Guo et al., 2024b). However, because most pretrained LLMs are trained on texts only, and developers can define any tools, most tools are out-of-domain (OOD) for LLMs. Even for the most powerful models, failures often happen when calling the newly defined tools due to a wrong calling format or failed parameter parsing. Thus, training an LLM to master a newly defined tool is critical for LLM agents. Intuitively, the tool-calling trajectories can be generated by a more powerful LLM, and such trajectories can be used to train a student model via supervised finetuning (SFT) (Chen et al., 2025c). However, this pipeline requires a stronger teacher model that has the capability to master the tool. Recently, more methods have emerged with no teacher model required. Rejected-sampled finetuning (RFT) (Ahn et al., 2024) utilizes generated trajectories of the agent itself via multiple rollouts. Agentic RL (Jin et al., 2025a) is an on-policy RLVR method requiring only the result for verifying trajectories. Such training methods yield remarkable results when the tools are search engines (Jin et al., 2025a), Python executors (Jimenez et al., 2023), calculators (Yan et al., 2025), and visual models (Gupta & Kembhavi, 2023).

### 2.2. Software Engineering Agents

The introduction of SWE-bench (Jimenez et al., 2023; Yang et al., 2024b) has motivated a range of agentic pipelines for software engineering tasks. Among them, SWE-agent (Yang et al., 2024a) and OpenHands (Wang et al., 2025b) are widely adopted frameworks that equip agents with tools for interacting with computing environments (terminals in Linux). Workflow-based methods such as Agentless (Xia et al., 2024) decompose issue resolution into localization, repair, and validation subproblems.

While DeepSWE (Luo et al., 2025) and SWE-Swiss (He et al., 2025) achieve high performance through reinforcement learning, end-to-end training remains computationally expensive due to the overhead of executing Docker containers for patch evaluation. Consequently, issue localization has emerged as a lightweight alternative, focusing on identifying faulty files or functions. Empirical evidence suggests that superior localization significantly improves overall issue resolution efficiency (Chen et al., 2025c; Xia et al., 2024; Yu et al., 2025b; Ma et al., 2025).

Recent localization agents include: LocAgent (Chen et al., 2025c), CoSIL (Jiang et al., 2025), and GraphLocator (Liu et al., 2025a), which model codebases as graphs and integrate them into LLMs, and OrcaLoca (Yu et al., 2025b), which enhances efficiency through priority scheduling, ac-

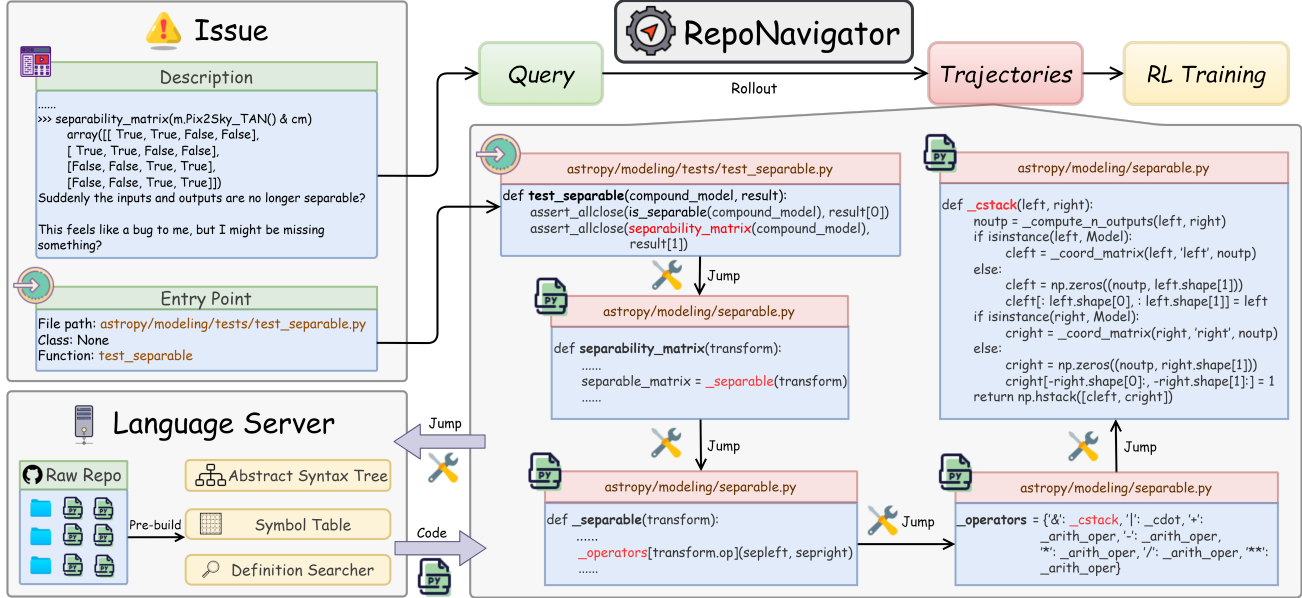


Figure 2. Overview of our RepoNavigator. During the rollout phase, the agent can invoke the `jump` tool, and the language server will return the definition code of the symbol. This process is trained by reinforcement learning.

tion decomposition, and context pruning. From an open-source perspective, RepoSearcher (Ma et al., 2025), trained with distillation and RL on the Qwen model family (Team, 2024), represents a notable advancement.

Nevertheless, prior agents overlook the structural relations within repositories—where modules, classes, and functions are cross-referenced across files—and typically rely on multiple search tools for symbol definition retrieval, amplifying error propagation (see Sec. 3). In contrast, we employ a single execution-logic-focused tool, reducing usage complexity. Finally, our approach constitutes the first localization agent trained directly from pretrained models, without relying on distillation-based supervised finetuning, a crucial stage in both RepoSearcher (Ma et al., 2025) and LocAgent (Chen et al., 2025c).

### 3. Method

We present **RepoNavigator**, a reinforcement-learning agent for repository-level issue localization. The method consists of three components: (1) a unified tool to retrieve the definition of any symbols in a given file, (2) a reasoning–action agent loop that alternates between natural-language reasoning and tool invocation, and (3) a GRPO-based RL algorithm for optimizing long-horizon tool-augmented trajectories.

#### 3.1. Problem Formulation

Given a repository  $\mathcal{R} = \{f_1, \dots, f_N\}$  and an issue description  $q$ , the goal is to output relevant code regions  $Y^* = \{(f_i, g_{i,j})\}$ , where  $g_{i,j}$  denotes a function or code

span in file  $f_i$ . At each step  $t$ , the agent produces an optional reasoning step  $r_t$ , a tool call  $a_t$ , and receives the observation  $o_t$ , forming a trajectory  $\tau = \{(r_t, a_t, o_t)\}_{t=1}^T$ . After termination, a final prediction  $\hat{Y}$  is scored by a reward  $R(\hat{Y}, Y^*)$ . The objective is  $\max_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau)]$ .

#### 3.2. Agent Architecture

RepoNavigator uses a *single-tool* design to avoid multi-tool orchestration overhead. At each step, the policy  $\pi_{\theta}$  decides whether to continue reasoning or to emit a JSON-formatted tool call, while a symbol and its corresponding file are parsed to the tool. The agent receives structured observations (code snippets or error messages), then continues reasoning until termination. The loop is *reason*  $\rightarrow$  *act*  $\rightarrow$  *observe* (Yao et al., 2022). Given history  $h_t = (q, o_{1:t-1}, a_{1:t-1})$ , the agent samples either a natural-language reasoning step  $r_t \sim \pi_{\theta}(\cdot | h_t)$  or a structured tool call  $a_t \sim \pi_{\theta}(\cdot | h_t)$ . Tool calls must satisfy a JSON grammar enforced via constrained decoding. The loop continues until the agent predicts its final locations  $\hat{Y}$ .

#### 3.3. Jump: Symbol Resolution

Language servers resolve the definition of a symbol via a deterministic static analysis pipeline that approximates runtime name-binding semantics. Given a symbol occurrence  $s$  at source location  $\ell$ , the resolution process computes a mapping from the source location to the definition location:

$$\mathcal{R}(s, \ell) \rightarrow \{(f_i, p_i)\}, \quad (1)$$

where each pair  $(f_i, p_i)$  denotes a file path and source position corresponding to a valid definition of  $s$ . In practice, we provide `file_path` and `symbol` to locate  $\ell$ ; when multiple symbols share the same name within a snippet, an additional `index` is used to disambiguate the targets.

**Syntactic and Scope Analysis** The source file is first parsed into an abstract syntax tree (AST). The syntactic role of  $s$  (e.g., name or attribute access) determines the resolution strategy. For name symbols, candidate definitions are searched along the lexical scope chain

$$\mathcal{S} = \{\text{local}, \text{enclosing}, \text{module}, \text{builtins}\}, \quad (2)$$

following the LEGB rule (for languages like Python, C++, etc.), with each scope maintaining a symbol table.

**Type-Based Attribute Resolution** For attribute expressions  $a.b$ , the language server infers a (possibly union-valued) type for the receiver  $a$  using annotations, control-flow analysis, function return types, and stub files. Member lookup is then performed over the method resolution order (MRO), denoted as:

$$\text{resolve}(a.b) = \bigcup_{t \in T(a)} \text{lookup}(b, \text{MRO}(t)). \quad (3)$$

**Import Dependency Graph** Cross-file resolution relies on an import dependency graph that statically emulates the module loading semantics. Import statements introduce bindings from local names to exported symbols of target modules, including re-exports and `__all__`-based filtering, and resolution may traverse multiple modules before reaching a concrete definition.

**Role of Jump** The jump operation is not an oracle that directly reveals task-relevant code. Rather, it abstracts execution-aware navigation already provided by modern IDE language servers for almost all programming languages. Consequently, the agent still faces a long-horizon decision problem: selecting which jump sequences to execute and when to stop exploration. We address this by training the agent with reinforcement learning in the next section.

### 3.4. Reinforcement Learning

We apply reinforcement learning with verifiable rewards to train the agent directly from the pretrained model. We do not require a more powerful teacher model for distillation as a warmup, which is the technique of (Ma et al., 2025). In practice, we apply Group Relative Policy Optimization (GRPO) (Liu et al., 2024a), which has the loss function:

$$\mathcal{L}^{\text{GRPO}}(\theta) = \mathbb{E}_{(s_t, a_t) \sim \pi_{\theta_{\text{old}}}} \left[ \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t - \beta D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot | s_t) \| \pi_{\theta}(\cdot | s_t)) \right] \quad (4)$$

where the first term is the standard policy gradient objective with an estimated advantage function  $\hat{A}_t$ , which promotes actions that lead to higher-than-expected returns. The second term is a Kullback-Leibler (KL) divergence penalty, scaled by a coefficient  $\beta$ , which acts as a trust region, preventing the updated policy  $\pi_{\theta}$  from moving too far from the previous policy  $\pi_{\theta_{\text{old}}}$ . This formulation ensures stable and consistent policy improvement by balancing reward maximization with behavioral consistency. The advantage  $\hat{A}_t$  is calculated in the standard GRPO method, and Algorithm 1 presents the full RL training process of RepoNavigator.

The reward of the GRPO process is calculated as:

$$R(\hat{Y}, Y^*, \tau) = \lambda \cdot \text{DICE}(\hat{Y}, Y^*) + (1 - \lambda) \cdot S(\tau) \quad (5)$$

where  $\lambda$  is heuristically set to 0.5 to balance the tool reward and the outcome reward (we prove the insensitivity of our method to  $\lambda$  in Appendix C.4). For the predicted set  $\hat{Y}$ , and the groundtruth set  $Y^*$

$$\text{DICE}(\hat{Y}, Y^*) = \frac{2 \times |\hat{Y} \cap Y^*|}{|\hat{Y}| + |Y^*|} \quad (6)$$

and  $S(\tau)$  is the success rate of tool-calling extracted from  $\tau$ . We consider the tool call to be failed when the format is incorrect, or the symbol parsed does not exist, or for any other reason that results in unexpected tool termination.

## 4. Experiment

### 4.1. Experiment Setup

**Datasets** We extract valid samples from SWE-smith (Yang et al., 2025b) to form the training set. We apply Qwen2.5-7B-Instruct with RepoNavigator to sample each data 16 times. A sample is abandoned if all 16 scores are zero. For validation, we test our method on SWE-bench\_Verified (Jimenez et al., 2023), and the Python subset of SWE-bench\_Pro (Yang et al., 2025b). For ground-truth locations, we directly use the locations in golden patches. All datasets are open-source and are built on real-world GitHub issues. We add an entry point to the user prompt (for RepoNavigator and all baselines) for each issue as the starting context; details are in Appendix B.

**Metrics** Previous works (Chen et al., 2025c; Ma et al., 2025) applied recall and precision as metrics. However, because the predicted locations and ground-truth locations

# One Tool Is Enough: Reinforcement Learning of LLM Agents for Repository-Level Code Navigation

Table 1. Comparison of different agent pipelines on function-level and file-level metrics on SWE-bench\_Verified. We use Qwen2.5-Instruct series as our base model. **Bold numbers** denote the best performance among same-size models; underline numbers denote the best training-free performance among same-size models; **yellow background** illustrates training-free RepoNavigator; **blue background** illustrates RepoNavigator trained with GRPO.

| Agent Pipeline       | Model             | Function-level |              |              |              | File-level   |              |              |              |
|----------------------|-------------------|----------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
|                      |                   | Recall         | Precision    | Sample-F1    | IoU          | Recall       | Precision    | Sample-F1    | IoU          |
| closed-source Models |                   |                |              |              |              |              |              |              |              |
| RepoSearcher         | Claude3.7-Sonnet  | <b>66.80</b>   | 19.90        | 28.30        | 17.89        | <b>89.71</b> | 21.04        | 33.15        | 20.67        |
| RepoNavigator        | Claude3.7-Sonnet  | 31.03          | 34.43        | 31.72        | 30.22        | 72.26        | 75.95        | 73.01        | 71.37        |
| RepoNavigator        | GPT5-chat         | 30.42          | 34.56        | 31.17        | 29.67        | 58.17        | 61.87        | 58.88        | 57.33        |
| RepoNavigator        | Claude4.5-Sonnet  | 43.97          | <b>45.76</b> | <b>43.62</b> | <b>41.31</b> | 80.68        | <b>81.92</b> | <b>79.94</b> | <b>77.49</b> |
| Qwen2.5-7B           |                   |                |              |              |              |              |              |              |              |
| Agentless            | Training-free     | 24.92          | 12.93        | 15.31        | 11.74        | 63.01        | 19.32        | 27.82        | 18.85        |
| CoSIL                | Training-free     | <u>29.30</u>   | 8.98         | 12.90        | 8.07         | <u>70.12</u> | 17.90        | 27.39        | 17.42        |
| LocAgent             | Training-free     | 17.62          | 11.71        | 12.71        | 10.31        | 60.96        | 34.88        | 40.67        | 33.33        |
| OrcaLoca             | Training-free     | 27.70          | <u>20.29</u> | <u>21.70</u> | <u>17.92</u> | 48.04        | <u>48.65</u> | <u>47.36</u> | <u>45.77</u> |
| RepoSearcher         | Training-free     | 7.86           | 8.08         | 7.48         | 6.85         | 48.61        | 21.15        | 27.36        | 20.39        |
| RepoSearcher         | Distillation+GRPO | <b>63.26</b>   | 19.24        | 27.37        | 17.59        | <b>84.11</b> | 19.97        | 31.64        | 19.57        |
| RepoNavigator        | Training-free     | 15.89          | 17.46        | 16.19        | 15.46        | 42.36        | 43.23        | 42.12        | 40.97        |
| RepoNavigator        | GRPO              | 26.69          | <b>30.34</b> | <b>27.49</b> | <b>26.43</b> | 50.62        | <b>53.83</b> | <b>51.63</b> | <b>50.62</b> |
| Qwen2.5-14B          |                   |                |              |              |              |              |              |              |              |
| Agentless            | Training-free     | 25.20          | 14.30        | 16.14        | 12.28        | 75.65        | 19.76        | 29.88        | 19.30        |
| CoSIL                | Training-free     | <b>48.61</b>   | 13.40        | 19.81        | 12.12        | <b>78.35</b> | 18.10        | 28.79        | 17.72        |
| LocAgent             | Training-free     | 35.62          | 13.32        | 17.71        | 12.32        | 71.42        | 31.66        | 40.77        | 30.64        |
| OrcaLoca             | Training-free     | 29.92          | 20.98        | 22.77        | 18.92        | 52.17        | 52.15        | 50.93        | 48.72        |
| RepoSearcher         | Training-free     | 26.13          | 11.96        | 14.35        | 10.60        | 74.77        | 18.80        | 28.79        | 18.15        |
| RepoNavigator        | Training-free     | 27.96          | <u>25.77</u> | <u>25.58</u> | <u>23.00</u> | 59.00        | <u>56.68</u> | <u>56.39</u> | <u>53.74</u> |
| RepoNavigator        | GRPO              | 31.02          | <b>30.08</b> | <b>29.23</b> | <b>26.84</b> | 61.60        | <b>58.97</b> | <b>58.90</b> | <b>56.36</b> |
| Qwen2.5-32B          |                   |                |              |              |              |              |              |              |              |
| Agentless            | Training-free     | 40.79          | 24.07        | 27.33        | 22.08        | 78.93        | 25.60        | 35.38        | 24.96        |
| CoSIL                | Training-free     | 55.38          | 14.85        | 22.11        | 13.52        | 83.50        | 19.34        | 30.77        | 18.93        |
| LocAgent             | Training-free     | 46.79          | 16.29        | 21.48        | 14.18        | 79.39        | 34.18        | 44.18        | 33.24        |
| OrcaLoca             | Training-free     | 39.14          | <u>25.59</u> | <u>28.72</u> | 22.89        | 59.57        | 59.51        | 58.11        | 55.62        |
| RepoSearcher         | Training-free     | 18.83          | 19.05        | 18.10        | 16.79        | 66.60        | 39.81        | 45.81        | 37.93        |
| RepoSearcher         | Distillation+GRPO | <b>69.50</b>   | 20.29        | 29.11        | 18.23        | <b>89.33</b> | 20.27        | 32.93        | 20.35        |
| RepoNavigator        | Training-free     | 28.11          | 28.19        | 27.12        | <u>25.16</u> | 63.05        | <u>62.75</u> | <u>61.67</u> | <u>59.28</u> |
| RepoNavigator        | GRPO              | 33.71          | <b>37.19</b> | <b>34.09</b> | <b>32.30</b> | 67.29        | <b>70.76</b> | <b>67.75</b> | <b>65.75</b> |

are sets of strings, recall and precision singularly cannot reflect the performance fairly. Thus, we utilize sample-F1 (which is the averaged score of per-sample F1 values) and IoU (intersection over union) as our core metrics. At the same time, we also present the recall and precision scores to align with previous methods, although they do not reflect the methods' performance fairly.

**Training** For the 7B model, we conduct GRPO with 8 NVIDIA Tesla-A100-80G GPUs. For the 14B and 32B models, we train them with 16 NVIDIA Tesla-A100-80G GPUs. We apply verl (Shen, 2024) as the training framework, and we apply vLLM (Kwon et al., 2023) as the inference engine. We train the model for 1 epoch, while the training batch size is fixed to 128 on 4k training samples filtered from SWE-smith, with maximum prompt length and

## One Tool Is Enough: Reinforcement Learning of LLM Agents for Repository-Level Code Navigation

Table 2. Comparison of different agent pipelines on function-level and file-level metrics on SWE-bench\_Pro. **Bold numbers** denote the best performance among same-size models; underline numbers denote the best training-free performance among same-size models; yellow background illustrates training-free RepoNavigator; blue background illustrates RepoNavigator trained with GRPO.

| Agent Pipeline | Model         | Function-level |              |              |              | File-level   |              |              |              |
|----------------|---------------|----------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
|                |               | Recall         | Precision    | Sample-F1    | IoU          | Recall       | Precision    | Sample-F1    | IoU          |
| Qwen2.5-7B     |               |                |              |              |              |              |              |              |              |
| Agentless      | Training-free | 12.82          | 6.94         | 8.05         | 5.73         | <b>39.41</b> | 13.15        | 18.89        | 12.35        |
| CoSIL          | Training-free | 8.64           | 3.33         | 4.58         | 2.87         | 26.64        | 8.47         | 12.11        | 7.70         |
| LocAgent       | Training-free | 1.01           | 0.02         | 0.65         | 0.40         | 12.16        | 0.17         | 10.81        | 8.93         |
| LocAgent       | Distillation  | <b>13.76</b>   | 1.61         | 2.66         | 1.48         | 37.44        | 5.11         | 8.39         | 5.00         |
| RepoSearcher   | Training-free | 1.07           | 0.93         | 0.97         | 0.86         | 4.91         | 1.64         | 2.30         | 1.63         |
| RepoNavigator  | Training-free | 9.84           | <u>14.65</u> | <u>10.67</u> | <u>9.20</u>  | 30.50        | <u>37.24</u> | <u>31.86</u> | <u>28.82</u> |
| RepoNavigator  | GRPO          | 12.33          | <b>21.26</b> | <b>14.29</b> | <b>12.02</b> | 36.36        | <b>48.13</b> | <b>39.74</b> | <b>36.36</b> |
| Qwen2.5-14B    |               |                |              |              |              |              |              |              |              |
| Agentless      | Training-free | 10.49          | 6.75         | 7.41         | 5.28         | 41.42        | 13.42        | 19.02        | 12.37        |
| CoSIL          | Training-free | 10.73          | 4.67         | 5.96         | 3.94         | 34.31        | 9.97         | 14.81        | 9.30         |
| LocAgent       | Training-free | 6.22           | 0.13         | 3.65         | 2.65         | 15.58        | 0.21         | 11.69        | 9.53         |
| RepoSearcher   | Training-free | 2.79           | 1.38         | 1.69         | 1.14         | 17.37        | 5.17         | 7.60         | 4.84         |
| RepoNavigator  | Training-free | <u>14.36</u>   | <u>19.74</u> | <u>15.27</u> | <u>12.00</u> | <u>43.57</u> | <u>54.52</u> | <u>46.06</u> | <u>41.07</u> |
| RepoNavigator  | GRPO          | <b>16.05</b>   | <b>25.25</b> | <b>18.06</b> | <b>14.58</b> | <b>46.85</b> | <b>58.64</b> | <b>49.72</b> | <b>45.14</b> |
| Qwen2.5-32B    |               |                |              |              |              |              |              |              |              |
| Agentless      | Training-free | 11.08          | 7.31         | 7.98         | 5.80         | 43.07        | 13.89        | 20.07        | 13.11        |
| CoSIL          | Training-free | <u>14.03</u>   | 6.00         | 7.67         | 4.92         | 48.87        | 14.03        | 20.95        | 13.27        |
| LocAgent       | Training-free | 8.72           | 0.17         | 4.30         | 2.90         | 25.73        | 0.38         | 19.77        | 16.50        |
| RepoSearcher   | Training-free | 2.52           | 2.46         | 2.31         | 1.75         | 9.00         | 2.52         | 3.81         | 2.41         |
| RepoNavigator  | Training-free | 13.96          | <u>20.25</u> | <u>15.36</u> | <u>12.87</u> | <u>50.24</u> | <u>63.24</u> | <u>53.48</u> | <u>48.50</u> |
| RepoNavigator  | GRPO          | <b>18.13</b>   | <b>29.44</b> | <b>20.72</b> | <b>17.16</b> | <b>53.49</b> | <b>68.69</b> | <b>57.57</b> | <b>52.44</b> |

max response length both set to 10240. Additionally, the group size  $G$  is set to 8, and the temperature is set to 1.0 to encourage exploration. We use greedy decoding in the inference stage to ensure stable performance. More experimental details are provided in Appendix B.

### 4.2. Effectiveness

**Baselines** We compare our method against LocAgent (Chen et al., 2025c), CoSIL (Jiang et al., 2025), Agentless (Xia et al., 2024), OrcaLoca (Yu et al., 2025b), and RepoSearcher (Ma et al., 2025). LocAgent is distilled from Claude3.5-Sonnet, and RepoSearcher is distilled from Claude3.7-Sonnet. See Appendix A for more details.

**Results** As illustrated in Table 1, on balanced metrics (Sample-F1 and IoU) for both function-level and file-level localization, our method surpasses all baseline methods with the same model size. Moreover, if we train RepoNavigator with GRPO, the 7B model surpasses 14B baselines, and our 14B model surpasses 32B baselines on Sample-F1 and IoU.

This further validates the effectiveness of RepoNavigator. For 14B and 32B models, RepoNavigator achieves SOTA among all training-free methods. This implies that the tool we implement is effective in practice, and our single tool pipeline is better than previous multi-tool pipelines.

Compared to RepoSearcher, which is distilled from claude-3.7-sonnet (Anthropic, 2025) and reinforced by GRPO, trained RepoNavigator outperforms it in all metrics except recall. Moreover, we found that our training-free method outperforms RepoSearcher for 14B models. We attribute this improvement to the simplified tool we integrate into the agent (see Sec. 5 for more details).

To assess the generalizability of RepoNavigator, we present its performance on Python samples from the SWE-bench\_Pro dataset (Yang et al., 2025b) in Table 2. The results on this dataset are consistent with those observed on SWE-bench\_Verified. While we cannot fully exclude the potential influence of data leakage in SWE-bench\_Verified, we can make a stronger claim regarding SWE-bench\_Pro, as it was released after the publication of the Qwen2.5 series.

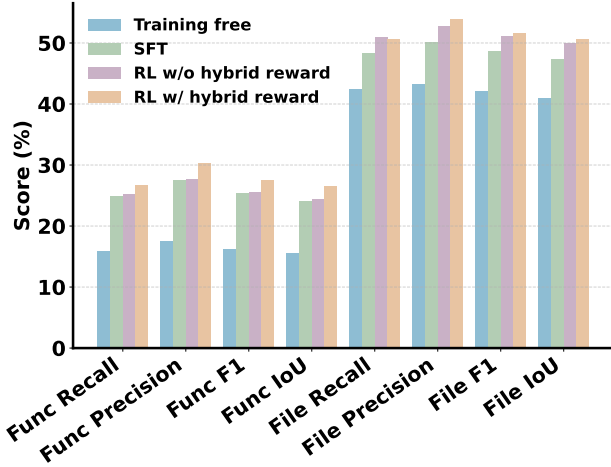


Figure 3. Training strategies comparison: comparison between RepoNavigator with training-free, RFT, GRPO with pure outcome and hybrid reward on Qwen2.5-7B-Instruct.

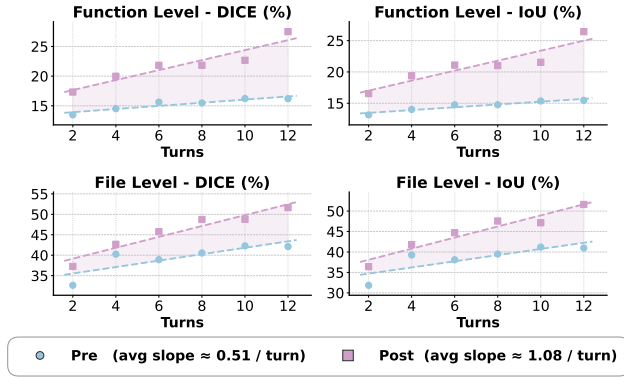


Figure 4. Scaling law of tool-calling, where *Pre* and *Post* denote the corresponding metric before and after training.

### 4.3. Training Strategy Comparison

We investigate the capabilities of GRPO and our proposed hybrid reward function within the context of agentic training, using SFT as a primary benchmark. Experimental results presented in Fig. 3 indicate that our method significantly surpasses both SFT and GRPO with a vanilla reward function, validating the critical role of RL and hybrid reward mechanisms in enhancing agentic capabilities. While (Ma et al., 2025) emphasizes the necessity of an initial SFT phase, our ablation study (detailed in Appendix C.1) reveals the potential of training fully autonomous agents directly through RL. This observation challenges the requirement for a “cold start” and resonates with a growing body of literature advocating for RL-centric training paradigms (Liu et al., 2024a; Kirk et al., 2023; Jin et al., 2025b).

We also remove the success rate in the reward function for ablation. As presented in Fig. 3, reinforcement learning with a hybrid reward (with tool-calling success rate) has higher

Table 3. Downstream influence: We use Qwen2.5-14B-Instruct on SWE-bench\_Verified to evaluate issue resolution.

| Agent Pipeline   | Func-IoU(%) | Resolved(%) |
|------------------|-------------|-------------|
| Agentless        | 12.28       | 10.12       |
| LocAgent         | 12.32       | 13.01       |
| RepoSearcher     | 10.60       | 12.47       |
| RepoNavigator    | 23.00       | 14.74       |
| RepoNavigator+RL | 26.84       | 15.03       |

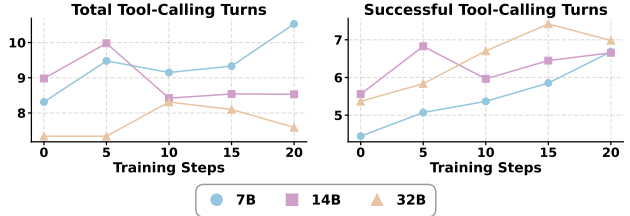


Figure 5. Training dynamics of tool-calling behaviors.

performance than pure outcome reward (without tool-calling success rate). This indicates that learning to correctly call tools is vital in agentic learning.

### 4.4. Scaling Law of Tool-Calling

To assess the significance of tool-calling in RepoNavigator, we varied the maximum number of tool-calling turns and reported the results in Fig. 4. As shown in the figure, allowing more tool-calling turns generally leads to improved performance for RepoNavigator, both before and after reinforcement learning (RL) training. These results empirically validate the scaling law of tool-calling in this context.

### 4.5. Evolution of tool-calling proficiency

A potential side effect of incorporating success rates into the GRPO reward function is the emergence of conservative agents that artificially inflate success metrics by suppressing tool usage. However, the empirical results in Fig. 5 alleviate this concern. Specifically, the number of successful tool-calling turns increases, while the total number of turns remains stable or even rises. For the tool success rate, Fig. 7 in the Appendix illustrates a rising trend. Notably, the 7B model exhibits a pronounced increase in total turns. In contrast to the 14B and 32B models—which leverage stronger reasoning capabilities to reach targets through fewer but more precise tool invocations—the 7B model undergoes a more extensive exploration phase. This behavior can be attributed to its initially weaker tool mastery, as evidenced by the lower tool success rates shown in Fig. 7 in the Appendix.

### 4.6. Influence on Issue Resolution

To assess how different localization strategies affect downstream issue resolution, we evaluate RepoNavigator against

baseline methods on SWE-bench\_Verified. Specifically, we adopt the repair phase of Agentless and substitute its localization module with alternative approaches. The results are summarized in Table 3. RepoNavigator consistently outperforms all baselines in terms of issue resolution accuracy, and the incorporation of reinforcement learning further enhances its performance.

## 5. Discussion: Building Fewer yet More Capable Tools

In this section, we analyze the logic behind RepoNavigator: A minimalist, unified toolset is more effective than a multitude of fragmented, narrowly scoped tools.

### 5.1. Impact on the Action Space of Agents

Let the total number of available tools be denoted as  $k$ . When only a single tool—specifically the `jump` tool—is retained, the system’s structural relations become simpler, as both the action space and the observation space are restricted to what this tool can access. In this case, the set of possible actions and observable elements is smaller than when multiple tools are available. This reduction is generally beneficial, since additional tools often introduce new and unfamiliar interfaces that large language models have not been exposed to during pretraining.

### 5.2. Impact on Tool-Calling Success Rate

For a given process in issue localization (for instance, checking the code snippet of a function), let the success probability of the  $i$ -th call be  $p_i$ . For a task that requires  $k$  sequential tool calls, the overall success rate can be expressed as:

$$P_{\text{succ}}(k) = \prod_{i=1}^k p_i. \quad (7)$$

The cumulative success rate typically decreases as the number of required tool calls increases. Therefore, in general, completing a task with a single, more versatile tool tends to be more reliable than relying on multiple narrow-scope tools executed in sequence.

### 5.3. Impact on the Prediction Space

The access scope of a tool is defined as the complete set of files, symbols, and other resources that the tool can access within a repository. For a `jump` tool that navigates to symbol definitions, its access scope can be obtained by starting from a given entry point and recursively resolving all referenced symbols until no new definitions can be reached. Apparently, its access scope is significantly smaller than the full repository scope. Consequently, when computing the Intersection over Union (IoU) between the prediction set

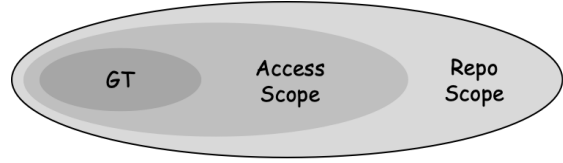


Figure 6. Venn graph illustrating the access scope of `jump`. Compared with the repository scope, the access scope of `jump` has a much higher IoU with the groundtruth set.

Table 4. We change the tool set of RepoNavigator and present the function-level IoU (%) on Qwen2.5-7B-Instruct. From the table, we demonstrate that excessive tools do not boost RepoNavigator’s performance.

| Jump | GetClass | GetFunc | GetStruc | IoU   |
|------|----------|---------|----------|-------|
| ✓    | ✓        | ✓       | ✓        | 13.71 |
| ✓    | ✓        | ✓       | ✗        | 21.44 |
| ✓    | ✗        | ✗       | ✓        | 24.00 |
| ✓    | ✗        | ✗       | ✗        | 24.28 |

and the groundtruth set, using the `jump` tool results in a higher IoU, as depicted in Fig. 6. On the other hand, applying multiple repo-level retrieval tools results in the access scope equal to the whole repository scope.

Starting from a designated entry point, the recursive application of the `jump` operation—which resolves the definitions of all referenced symbols—effectively constructs a transitive closure of all semantically activated symbols. Since any code location contributing to a specific issue must inherently reside on a dependency path originating from the entry point, it is logically reachable through this recursive expansion of symbol references. Consequently, the final access scope generated by an exhaustive `jump` traversal is guaranteed to encompass all locations requiring modification to resolve the issue, excluding only those introduced by dynamic, runtime changes to the repository (e.g., monkey-patching).

### 5.4. Verification

To further validate this hypothesis, we modify the tool set of RepoNavigator and perform reinforcement learning with only outcome-based rewards. We add additional tools commonly used in prior work (Chen et al., 2025c; Ma et al., 2025; Jiang et al., 2025). As shown in Table 4 (full results in Table 6), additional tools hinders performance. This suggests that **fewer yet more capable tools** are preferable.

## 6. Conclusion

In this work, we propose RepoNavigator, a repository-level issue localization agent that replaces complex multi-tool pipelines with a single, more capable `jump` tool for symbol resolution. This unified design better reflects real code execution flow while reducing the brittleness of tool chain-

ing. Trained with tool-integrated GRPO, RepoNavigator jointly learns reasoning and tool invocation in a closed loop, enabling end-to-end optimization without closed-source teacher models or distillation.

Extensive experiments across SWE-bench\_Verified and SWE-bench\_Pro demonstrate that RepoNavigator achieves state-of-the-art localization performance. We theoretically analyze the results, confirming that a single powerful tool, jointly optimized with reinforcement learning, can provide stronger robustness and more reliable multi-step reasoning than previous frameworks relying on multiple narrowly scoped tools. Future work will explore extending RepoNavigator from Python to more programming languages.

## Acknowledgement

This work is supported by the Zhongguancun Academy (Grant No.s C20250210). We thank Chenming Tang (Peking University) for the constructive suggestions provided during the discussion with the authors.

## Impact Statement

This paper presents RepoNavigator, an LLM agent designed to advance repository-level issue localization by introducing a streamlined, single-tool navigation paradigm and an autonomous Reinforcement Learning training framework that enhances the efficiency and accessibility of automated software maintenance. While this technology significantly reduces manual debugging overhead and democratizes complex codebase navigation, its precision in identifying functional weak points could potentially be repurposed for malicious intent, such as accelerating the discovery of software vulnerabilities for targeted cyberattacks. Furthermore, its integration into enterprise environments necessitates rigorous compliance frameworks to ensure that all AI-driven code modifications are meticulously tracked, auditable, and aligned with security standards to mitigate technical debt or legal liabilities.

## References

- Ahn, J., Verma, R., Lou, R., Liu, D., Zhang, R., and Yin, W. Large language models for mathematical reasoning: Progresses and challenges. *arXiv preprint arXiv:2402.00157*, 2024.
- Anthropic. Claude 3.7 sonnet and claude code. <https://www.anthropic.com/news/claude-3-7-sonnet>, February 2025. date: 2025-11-18.
- Chen, J., Yu, T., Bai, H., Yao, L., Wu, J., Li, K., Mi, F., Tao, C., Zhu, L., Zhang, M., et al. The synergy dilemma of long-cot sft and rl: Investigating post-training techniques for reasoning vlms. *arXiv preprint arXiv:2507.07562*, 2025a.
- Chen, L., Han, X., Shen, L., Bai, J., and Wong, K.-F. Beyond two-stage training: Cooperative sft and rl for llm reasoning. *arXiv preprint arXiv:2509.06948*, 2025b.
- Chen, P., Zhang, S., and Han, B. Comm: Collaborative multi-agent, multi-reasoning-path prompting for complex problem solving. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pp. 1720–1738, 2024.
- Chen, Z., Tang, R., Deng, G., Wu, F., Wu, J., Jiang, Z., Prasanna, V., Cohan, A., and Wang, X. LocAgent: Graph-guided LLM agents for code localization. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8697–8727, Vienna, Austria, July 2025c. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.426. URL <https://aclanthology.org/2025.acl-long.426/>.
- Gao, P., Tian, Z., Meng, X., Wang, X., Hu, R., Xiao, Y., Liu, Y., Zhang, Z., Chen, J., Gao, C., et al. Trae agent: An llm-based agent for software engineering with test-time scaling. *arXiv preprint arXiv:2507.23370*, 2025.
- Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X., Wu, Y., Li, Y., et al. Deepseek-coder: When the large language model meets programming—the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024a.
- Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., and Zhang, X. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*, 2024b.
- Gupta, T. and Kembhavi, A. Visual programming: Compositional visual reasoning without training. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 14953–14962, 2023.
- He, Z., Yang, Q., Sheng, W., Zhong, X., Zhang, K., An, C., Shi, W., Cai, T., He, D., Chen, J., and Xu, J. Swe-swiss: A multi-task fine-tuning and rl recipe for high-performance issue resolution. <https://github.com/zhenyuhe00/SWE-Swiss>, 2025. Notion Blog.
- Hong, W., Wang, W., Lv, Q., Xu, J., Yu, W., Ji, J., Wang, Y., Wang, Z., Dong, Y., Ding, M., et al. Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 14281–14290, 2024.

- Huang, X., Liu, W., Chen, X., Wang, X., Wang, H., Lian, D., Wang, Y., Tang, R., and Chen, E. Understanding the planning of llm agents: A survey. *arXiv preprint arXiv:2402.02716*, 2024.
- Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.
- Jiang, Z., Ren, X., Yan, M., Jiang, W., Li, Y., and Liu, Z. Cosil: Software issue localization via llm-driven code repository graph searching. *arXiv preprint arXiv:2503.22424*, 2025.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.
- Jin, B., Zeng, H., Yue, Z., Yoon, J., Arik, S., Wang, D., Zamani, H., and Han, J. Search-rl: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025a.
- Jin, H., Lv, S., Wu, S., and Hamdaqa, M. Rl is neither a panacea nor a mirage: Understanding supervised vs. reinforcement learning fine-tuning for llms. *arXiv preprint arXiv:2508.16546*, 2025b.
- Kirk, R., Mediratta, I., Nalmpantis, C., Luketina, J., Hambro, E., Grefenstette, E., and Raileanu, R. Understanding the effects of rlhf on llm generalisation and diversity. *arXiv preprint arXiv:2310.06452*, 2023.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- Langley, P. Crafting papers on machine learning. In Langley, P. (ed.), *Proceedings of the 17th International Conference on Machine Learning (ICML 2000)*, pp. 1207–1216, Stanford, CA, 2000. Morgan Kaufmann.
- Li, H., Tang, Y., Wang, S., and Guo, W. Patchpilot: A cost-efficient software engineering agent with early attempts on formal verification. *arXiv preprint arXiv:2502.02747*, 2025.
- Li, Y., Wen, H., Wang, W., Li, X., Yuan, Y., Liu, G., Liu, J., Xu, W., Wang, X., Sun, Y., et al. Personal llm agents: Insights and survey about the capability, efficiency and security. *arXiv preprint arXiv:2401.05459*, 2024.
- Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024a.
- Liu, W., Peng, C., Gao, P., Liu, A., Zhang, W., Zhao, H., and Jin, Z. Graphlocator: Graph-guided causal reasoning for issue localization. *arXiv preprint arXiv:2512.22469*, 2025a.
- Liu, X., Lan, B., Hu, Z., Liu, Y., Zhang, Z., Wang, F., Shieh, M. Q., and Zhou, W. Codexgraph: Bridging large language models and code repositories via code graph databases. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 142–160, 2025b.
- Liu, Y., Gao, P., Wang, X., Liu, J., Shi, Y., Zhang, Z., and Peng, C. Marscode agent: Ai-native automated bug fixing. *arXiv preprint arXiv:2409.00899*, 2024b.
- Liu, Z., Zhang, Y., Li, P., Liu, Y., and Yang, D. A dynamic llm-powered agent network for task-oriented agent collaboration. In *First Conference on Language Modeling*, 2024c.
- Lu, J., Holleis, T., Zhang, Y., Aumayer, B., Nan, F., Bai, F., Ma, S., Ma, S., Li, M., Yin, G., et al. Toolsandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities. *arXiv preprint arXiv:2408.04682*, 2024.
- Luo, M., Jain, N., Singh, J., Tan, S., Patel, A., Wu, Q., Ariyak, A., Cai, C., Tarun Venkat, S. Z., Athiwaratkun, B., Roongta, M., Zhang, C., Li, L. E., Popa, R. A., Sen, K., and Stoica, I. Deepswt: Training a state-of-the-art coding agent from scratch by scaling rl, 2025. URL <https://pretty-radio-b75.notion.site/DeepSWE-Training-a-Fully-Open-sourced-State-of-the-Art-Coding-Agent-by-Scaling-RL-22281902c1468193aabb9a8c59bbe33>. Notion Blog.
- Ma, Z., Peng, C., Zeng, Q., Gao, P., Zou, Y., and Xie, B. Tool-integrated reinforcement learning for repo deep search, 2025. URL <https://arxiv.org/abs/2508.03012>.
- Microsoft Corporation. Pyright: Static type checker for python. <https://github.com/microsoft/pyright>, 2024.
- Pan, J., Wang, X., Neubig, G., Jaitly, N., Ji, H., Suhr, A., and Zhang, Y. Training software engineering agents and verifiers with swe-gym. *arXiv preprint arXiv:2412.21139*, 2024.
- Schmidgall, S., Su, Y., Wang, Z., Sun, X., Wu, J., Yu, X., Liu, J., Moor, M., Liu, Z., and Barsoum, E. Agent laboratory: Using llm agents as research assistants. *arXiv preprint arXiv:2501.04227*, 2025.

- Shen, Z. Llm with tools: A survey. *arXiv preprint arXiv:2409.18807*, 2024.
- Sohrabizadeh, A., Song, J., Liu, M., Roy, R., Lee, C., Raiman, J., and Catanzaro, B. Nemotron-cortexa: Enhancing llm agents for software engineering tasks via improved localization and solution diversity. In *Forty-second International Conference on Machine Learning*.
- Team, Q. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.
- Wang, J., Zan, D., Xin, S., Liu, S., Wu, Y., and Shen, K. Swe-mirror: Scaling issue-resolving datasets by mirroring issues across repositories. *arXiv preprint arXiv:2509.08724*, 2025a.
- Wang, X., Li, C., Wang, Z., Bai, F., Luo, H., Zhang, J., Jojic, N., Xing, E. P., and Hu, Z. Promptagent: Strategic planning with language models enables expert-level prompt optimization. *arXiv preprint arXiv:2310.16427*, 2023.
- Wang, X., Li, B., Song, Y., Xu, F. F., Tang, X., Zhuge, M., Pan, J., Song, Y., Li, B., Singh, J., Tran, H. H., Li, F., Ma, R., Zheng, M., Qian, B., Shao, Y., Muennighoff, N., Zhang, Y., Hui, B., Lin, J., Brennan, R., Peng, H., Ji, H., and Neubig, G. Openhands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations*, 2025b. URL <https://openreview.net/forum?id=OJd3ayDDoF>.
- Wang, Y., Mao, W., Wang, C., Zhou, Z., Zhou, Y., Zhao, W., Lou, Y., and Peng, X. Extracting conceptual knowledge to locate software issues. *arXiv preprint arXiv:2509.21427*, 2025c.
- Xia, C. S., Deng, Y., Dunn, S., and Zhang, L. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.
- Xiang, D., Xu, W., Chu, K., Ding, T., Shen, Z., Zeng, Y., Su, J., and Zhang, W. Promptsulptor: Multi-agent based text-to-image prompt optimization. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 774–786, 2025.
- Xue, H., Uddin, G., and Wang, S. Pagent: Learning to patch software engineering agents. *arXiv preprint arXiv:2506.17772*, 2025.
- Yan, Y., Wang, S., Huo, J., Yu, P. S., Hu, X., and Wen, Q. Mathagent: Leveraging a mixture-of-math-agent framework for real-world multimodal mathematical error detection. *arXiv preprint arXiv:2503.18132*, 2025.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025a.
- Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K. R., and Press, O. SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024a. URL <https://arxiv.org/abs/2405.15793>.
- Yang, J., Jimenez, C. E., Zhang, A. L., Lieret, K., Yang, J., Wu, X., Press, O., Muennighoff, N., Synnaeve, G., Narasimhan, K. R., et al. Swe-bench multimodal: Do ai systems generalize to visual software domains? *arXiv preprint arXiv:2410.03859*, 2024b.
- Yang, J., Lieret, K., Jimenez, C. E., Wettig, A., Khandpur, K., Zhang, Y., Hui, B., Press, O., Schmidt, L., and Yang, D. Swe-smith: Scaling data for software engineering agents. *arXiv preprint arXiv:2504.21798*, 2025b.
- Yang, X., Zhou, J., Pacheco, M., Zhu, W., He, P., Wang, S., Liu, K., and Pan, R. Lingxi: Repository-level issue resolution framework enhanced by procedural knowledge guided scaling. *arXiv preprint arXiv:2510.11838*, 2025c.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.
- Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., Yue, Y., Dai, W., Fan, T., Liu, G., Liu, L., et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025a.
- Yu, Z., Zhang, H., Zhao, Y., Huang, H., Yao, M., Ding, K., and Zhao, J. Orcaloca: An llm agent framework for software issue localization, 2025b. URL <https://arxiv.org/abs/2502.00350>.
- Yuan, S., Song, K., Chen, J., Tan, X., Shen, Y., Kan, R., Li, D., and Yang, D. Easytool: Enhancing llm-based agents with concise tool instruction. *arXiv preprint arXiv:2401.06201*, 2024.
- Yue, Y., Yuan, Y., Yu, Q., Zuo, X., Zhu, R., Xu, W., Chen, J., Wang, C., Fan, T., Du, Z., et al. Vapo: Efficient and reliable reinforcement learning for advanced reasoning tasks. *arXiv preprint arXiv:2504.05118*, 2025.

## One Tool Is Enough: Reinforcement Learning of LLM Agents for Repository-Level Code Navigation

Table 5. Illustration of the tool sets of agentic methods (Agentless (Xia et al., 2024) is not included). Among all methods, only RepoNavigator takes the `jump` logic into consideration, and RepoNavigator has the smallest number of tools.

| Method       | Number | Tools                                                                                                   | Input                                                                                                                                         | Output                                                                                                                                                                                                               |
|--------------|--------|---------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CoSIL        | 3      | search_class_node<br>search_class_function_node<br>search_file_function_node                            | file_path, class_name<br>file_path, class_name, function_name<br>file_path, function_name                                                     | The code snippet of a class node.<br>Get the code snippet of a class member function node.<br>Get the code snippet of a static function node.                                                                        |
| LocAgent     | 3      | SearchEntity<br>TraverseGraph<br>RetrieveEntity                                                         | Keywords<br>Start entity ids, direction, traverse hops, entity types, relation types<br>Entity ids                                            | Related entities with code snippets.<br>A traversed subgraph with entities and relations.<br>The complete code of a specific entity.                                                                                 |
| OrcaLoca     | 5      | search_file_contents<br>search_class<br>search_method_in_class<br>search_callable<br>search_source_code | file_name, directory_path<br>class_name, method_name<br>class_name, method_name, file_path<br>query_name, file_path<br>file_path, source_code | The file contents or skeleton (if exceeds 200 lines).<br>The class contents or skeleton (if exceeds 200 lines).<br>The method code snippet.<br>The matched code snippet.<br>The related function/class code snippet. |
| RepoSearcher | 5      | GetRepoStructure<br>GetImportOfFile<br>SearchClass<br>SearchFunction<br>SearchClassMethod               | None<br>file, class<br>file, class<br>file, function<br>file, class, method                                                                   | The repository file structure.<br>The imports of file.<br>Code content of the searched class.<br>Code content of the searched function.<br>Code content of the searched method.                                      |
| Ours         | 1      | Jump                                                                                                    | file_path, symbol, index (optional)                                                                                                           | Definition code snippet of the symbol.                                                                                                                                                                               |

### A. Detailed Illustration of Baselines

The comprehensive toolsets for all aforementioned methods are detailed in Table 5. Furthermore, this section provides an in-depth discussion regarding the degree of autonomy exhibited by these approaches.

**Agentless** Agentless (Xia et al., 2024) is a workflow for issue localization. First, it identifies suspicious files in the repository. Second, relevant classes and functions are detected. Third, precise locations for editing are given by LLMs based on the classes and functions.

**CoSIL** CoSIL (Jiang et al., 2025) is an agent that first conducts file-level localization and then conducts function-level localization. CoSIL dynamically constructs call graphs of modules (classes, functions) during the repo-level searching process, and applies context pruning to effectively reduce the searching scope.

**LocAgent** LocAgent (Chen et al., 2025c) is almost a fully-automatic LLM agent, besides its planning prompt concatenated into the context at the beginning of the searching process. It builds the whole repository into a direct heterogeneous graph, whose nodes are files, classes, and functions. Additionally, edges are built by dependencies such as imports and invocations. Multiple graph-level searching tools are equipped to the LLM for multi-hop reasoning.

**OrcaLoca** OrcaLoca (Yu et al., 2025b) is an LLM-based agent framework for software issue localization that performs repository-level exploration via priority-based action scheduling. It incrementally decomposes file- and class-level searches into fine-grained function-level actions with relevance scoring, while applying distance-aware context pruning on a code graph to efficiently narrow down potential bug locations. Because we failed to reproduce the code of OrcaLoca, we did not evaluate it on SWE-ben\_Pro, and we directly report the results (provided by Jiang et al. (2025)) on SWE-bench\_Verified.

**RepoSearcher** RepoSearcher (Ma et al., 2025) is an agent that first conducts file-level localization and then function-level localization, which aligns with CoSIL. RepoSearcher introduced the first training framework *ToolTrain* for localization agents, which is composed of distilling from a closed-source model (Claude3.7-Sonnet in Reposearcher) as warmup and reinforcement learning to further enhance the performance.

**Ours** Compared with all baselines, we are the first fully-automatic LLM agent, with no fixed workflow and no planning prompt, and we are the first method trained directly from pretrained open-source LLMs without a closed-source teacher model. Lastly, we only integrate a single yet powerful tool to the agent, which reduces compounding error and narrows the access scope of the agent.

Table 6. Comparison of tool configurations for RepoNavigator. ✓ denotes having the tool, and ✗ denotes not having the tool. When Jump is equipped, excessive tools do not improve the performance.

| Jump | GetClass | GetFunc | GetStruc | Function-level |              |              |              |
|------|----------|---------|----------|----------------|--------------|--------------|--------------|
|      |          |         |          | Recall         | Precision    | Sample-F1    | IoU          |
| ✓    | ✓        | ✓       | ✓        | 14.28          | 15.44        | 14.40        | 13.71        |
| ✓    | ✓        | ✓       | ✗        | 22.60          | 25.02        | 22.80        | 21.44        |
| ✓    | ✗        | ✗       | ✓        | 24.64          | 27.48        | 25.05        | 24.00        |
| ✓    | ✗        | ✗       | ✗        | <b>25.11</b>   | <b>29.16</b> | <b>25.75</b> | <b>24.28</b> |

## B. Experimental Details

**Benchmarks** The SWE-bench (Jimenez et al., 2023) dataset series is designed to evaluate the capability of LLM agents in resolving real-world GitHub issues. In the official evaluation protocol, models are provided solely with the problem statement and the historical repository state to generate a patch. However, in practical software engineering workflows, developers typically have access to at least one entry point for any given issue. To better simulate these real-world scenarios and enhance evaluation fidelity, we deviate from the official SWE-bench standard by providing an explicit entry point for each sample. Specifically, for both our proposed method and all baseline models, we supply the precise file path and the corresponding entry function.

**Tools Description** Table 5 illustrates the detailed input and output of RepoNavigator and baselines (except Agentless, which is workflow-based). CoSIL, OrcaLoca, and RepoSearcher apply simple retrieval tools that overlook the structural information of the repository. Among all methods, only RepoNavigator reduces the number of tools to **one**.

**Hyperparameters for RL** We set `clip_ratio_low` to 0.2, `clip_ratio_high` to 0.8, learning rate to  $10^{-6}$ , `training_batch_size` to 128, temperature to 1.0, `maximum_toolcalling` to 12, and `max_response_length` to 10240.

**Hyperparameters for Baselines** For baseline methods, we use vLLM (Kwon et al., 2023) to deploy the corresponding model locally, and we set `max_response_length` to 32768, `enforce-eager` to True. We deploy all models on 8 NVIDIA Tesla-A100-80G GPUs. We follow the default settings of vLLM for other hyperparameters.

**Metrics** Given the set of predicted locations (either file-level or function-level)  $\hat{Y}$ , and the set of groundtruth locations  $Y^*$ , the aforementioned metrics are calculated as:

$$\text{Recall} = \frac{|\hat{Y} \cap Y^*|}{|Y^*|} \quad (8)$$

$$\text{Precision} = \frac{|\hat{Y} \cap Y^*|}{|\hat{Y}|} \quad (9)$$

$$\text{Sample-F1} = \frac{2 \times |\hat{Y} \cap Y^*|}{|\hat{Y}| + |Y^*|} \quad (10)$$

$$\text{IoU} = \frac{|\hat{Y} \cap Y^*|}{|\hat{Y} \cup Y^*|} \quad (11)$$

In practice, when the prediction set  $\hat{Y}$  is empty (for instance, total failure), we set recall, precision, sample-F1, and IoU to zero. We use the function-level localization result of different methods and apply the patch generation backend in Agentless (Xia et al., 2024) to generate patches. Resolved(%) denotes the percentage of samples that pass all test units after applying the patch in Table 3.

**Implementation** When the response exceeds the maximum length, we clip and force the agent to stop, and we give zero as its score. When the agent exceeds the maximum tool-calling times (which is 12), we add “**You must not call tools anymore, and you must give the final answer**” to the tool’s response. Most of the time, the agent will stop calling tools and generate

## One Tool Is Enough: Reinforcement Learning of LLM Agents for Repository-Level Code Navigation

Table 7. The model achieves competitive and steady scores as  $\lambda$  ranges from 0.4 to 0.6. This insensitivity suggests that the heuristic setting of  $\lambda = 0.5$  is near-optimal and the synergy between outcome and tool rewards is inherently stable, rather than being sensitive to specific weighting.

| $\lambda$ | Function-level |           |           |       | File-level |           |           |       |
|-----------|----------------|-----------|-----------|-------|------------|-----------|-----------|-------|
|           | Recall         | Precision | Sample-F1 | IoU   | Recall     | Precision | Sample-F1 | IoU   |
| 0.6       | 26.32          | 28.86     | 26.85     | 25.88 | 50.93      | 52.66     | 51.18     | 50.23 |
| 0.5       | 26.69          | 30.34     | 27.49     | 26.43 | 50.62      | 53.83     | 51.63     | 50.62 |
| 0.4       | 27.18          | 30.70     | 28.08     | 27.18 | 50.67      | 52.56     | 51.49     | 50.67 |

Table 8. Ablation study on SFT and RL.

| SFT | RL | Function-level |              |              |              |
|-----|----|----------------|--------------|--------------|--------------|
|     |    | Recall         | Precision    | Sample-F1    | IoU          |
| ✗   | ✗  | 15.89          | 17.46        | 16.19        | 15.46        |
| ✓   | ✗  | 24.92          | 27.44        | 25.38        | 24.01        |
| ✓   | ✓  | 25.93          | 30.08        | 26.97        | 25.75        |
| ✗   | ✓  | <b>26.69</b>   | <b>30.34</b> | <b>27.49</b> | <b>26.43</b> |

Table 9. Cost Analysis.

| Method        | Avg. Tokens ( $\times 10^3$ ) |        |       |
|---------------|-------------------------------|--------|-------|
|               | Input                         | Output | Total |
| Agentless     | 19.84                         | 2.36   | 22.20 |
| CoSIL         | 7.93                          | 0.50   | 8.44  |
| LocAgent      | 45.88                         | 2.98   | 48.86 |
| RepoSearcher  | 23.85                         | 2.81   | 26.66 |
| RepoNavigator | 1.73                          | 0.95   | 2.68  |

the final response. If not, we force it to stop and give zero as its score. Note that when the maximum tool-calling times are not achieved, and the final answer is generated, the agent loop will stop automatically. The aforementioned process is an automatic agentic framework, which allows the agent to explore environments with few constraints.

**Language Server** In practice, we apply a Python language server, Pyright (Microsoft Corporation, 2024), to extract the definition code corresponding to an invoked symbol within a repository. However, the presence of monkey patches—runtime modifications to the repository—and dynamic imports can degrade the performance of the language server, as its functionality relies on static analysis techniques such as abstract syntax trees and symbol tables. When such circumstances occur, the tool returns an error message indicating that the definition of the current symbol cannot be located due to unknown reasons. Nevertheless, in our empirical evaluation, we did not observe any instances of monkey patching or dynamic imports within the analyzed datasets.

## C. More Results

### C.1. Ablation Study

We investigate the impact of the hybrid reward mechanism in Fig. 3 and explore the influence of SFT as a “cold start” phase for the Reinforcement Learning (RL) process. As illustrated in Table 8, our findings suggest that while SFT enhances initial model performance, it ultimately constrains the long-term potential of RL. By forcing the model to adhere to pre-generated trajectories, SFT improves basic task proficiency at the expense of autonomous exploration. This finding aligns with previous research on this topic (Chen et al., 2025b; Jin et al., 2025b; Chen et al., 2025a). Consequently, the results in Table 8 demonstrate that training the model directly with reinforcement learning yields the most superior performance.

### C.2. Cost Analysis

We conduct a comparative analysis of token usage across multiple methodologies, utilizing Qwen2.5-14B-Instruct as the base model on SWE-bench\_Pro. Empirical results (see Table 9) indicate that RepoNavigator represents a superior agentic design in terms of cost-efficiency. While baseline approaches such as CoSIL and RepoSearcher incur substantial overhead by indexing all repository files—leading to significantly higher average input tokens—our proposed method optimizes token consumption by avoiding such exhaustive operations.

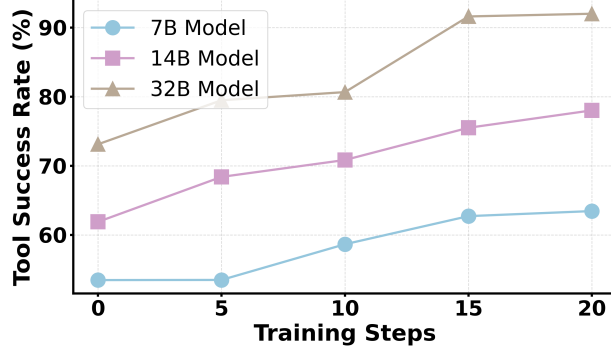


Figure 7. Tool success rate of the 7B, 14B, and 32B model.

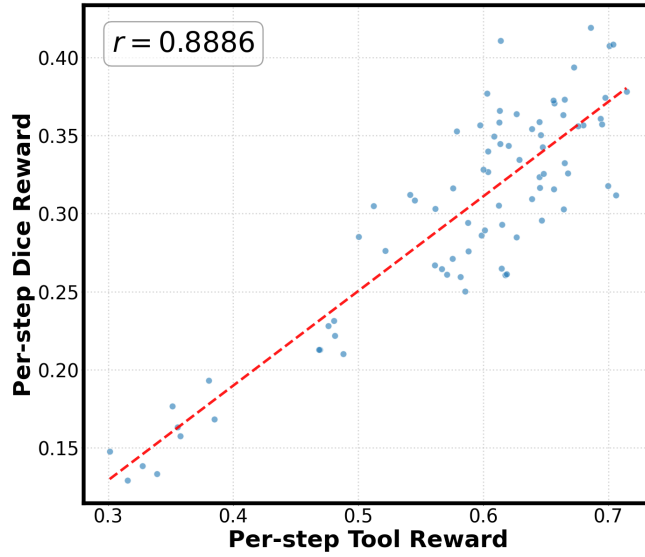


Figure 8. Evolution of the average tool reward during the training process. Higher rewards indicate improved efficiency in selecting and executing external tools.

### C.3. Rationality of the Joint Reward Mechanism.

The integration of tool reward and dice reward as a hybrid reinforcement learning objective is grounded in two key empirical observations. First, as illustrated in Fig. 7, the tool reward exhibits a consistent and stable upward trajectory during the training phase, demonstrating that the agent is effectively learning to master tool-use patterns and that this reward signal provides a dense, optimizable gradient. Second, Fig. 8 reveals a high Pearson correlation ( $r=0.8886$ ) between tool reward and dice reward. This strong correlation indicates that the intermediate tool-use proficiency is a reliable proxy for the final task success (as measured by the Dice score). By combining these two signals, the reward function not only captures the ultimate goal but also provides informative “stepping stones” during training, effectively mitigating the sparse reward problem while ensuring that the optimization of tool interaction aligns to improve overall performance.

### C.4. Sensitivity Analysis

In the design of our agent, we adopt a balanced heuristic by assigning equal weight to both outcome and tool rewards (i.e., a 1:1 ratio). This choice represents a non-informative prior, assuming that the correctness of the final result and the adherence to tool protocols are of equal importance. To evaluate the robustness of our method, we conduct a sensitivity study on the reward weighting factor  $\lambda$ . As illustrated in Table 7, the performance on the test set remains remarkably stable as  $\lambda$  varies.

## D. Detailed Analysis

### D.1. Jump as a Learnable Action Abstraction

We provide an analysis to explain why the proposed `Jump` tool constitutes a learnable and effective action abstraction for repository-level code navigation. We use Python as an example; other languages that are similar to Python (C++, Java...) are also suitable.

**Problem formulation.** We model repository navigation as a finite-horizon Markov Decision Process (MDP)  $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R)$ , where a state  $s \in \mathcal{S}$  represents the agent’s current context (including all historical conversations), and an action  $a \in \mathcal{A}$  corresponds to invoking a navigation tool. In prior approaches, the action space  $\mathcal{A}$  is composed of multiple heterogeneous tools (e.g., `SearchClass`, `SearchFunction`, `GetImports...`), each inducing distinct and noisy transitions. This results in a large effective branching factor and long-horizon exploration difficulty.

**Action abstraction via `Jump`.** Our method replaces the heterogeneous action space  $\mathcal{A}$  with a single abstract action space  $\mathcal{A}' = \{\text{Jump}(s)\}$ , where each action deterministically resolves a symbol to its definition using static analysis. Formally, this defines an abstraction mapping:

$$\phi : \mathcal{A} \rightarrow \mathcal{A}',$$

which collapses multiple low-level navigation actions into a single execution-aware transition. Importantly, this abstraction is *many-to-one* and preserves execution-relevant semantics, as symbol resolution directly reflects Python’s name-binding rules.

**Preservation of Reachability.** Although `Jump` does not have access to oracle ground-truth locations, it preserves the reachability of task-relevant program elements under realistic assumptions. Specifically, if a target symbol is statically resolvable within the repository, then there exists a sequence of `Jump` actions that reaches its definition. Consequently, the abstraction does not remove optimal solutions, but instead restricts exploration to semantically valid transitions.

**Reduction of Exploration Complexity.** The primary benefit of the abstraction lies in its effect on exploration. Let  $B$  denote the average branching factor induced by the original action space  $\mathcal{A}$ , and  $B'$  that induced by  $\mathcal{A}'$ . Because `Jump` deterministically resolves symbols and filters out irrelevant candidates, we empirically observe that:

$$\mathbb{E}[B'] \ll \mathbb{E}[B].$$

In long-horizon tasks, the probability of entering irrecoverable exploration states grows exponentially with the branching factor. Thus, reducing the branching factor substantially improves the tractability of reinforcement learning under finite rollouts.

**Implications for Policy Learning.** The abstraction reshapes the learning problem by aligning actions with the underlying program structure. As a result, the induced MDP  $\mathcal{M}' = (\mathcal{S}, \mathcal{A}', P', R)$  admits policies that are easier to optimize, even with standard policy-gradient methods. This explains why reinforcement learning from a pretrained language model succeeds in our setting without reliance on distillation from closed-source teacher LLMs.

In summary, `Jump` is not merely a tool simplification, but a principled action abstraction that preserves reachability while dramatically reducing exploration complexity, thereby enabling effective reinforcement learning for repository-level software engineering tasks.

### D.2. Necessity of Reinforcement Learning for Long-Horizon Repository Navigation

The objective of the agent is not merely to invoke the jump operation correctly, but to learn a policy for sequencing execution-level navigation steps under sparse terminal rewards. Intermediate states do not admit dense supervision, and optimal behavior depends on long-horizon credit assignment. Reinforcement learning provides a principled framework for optimizing such policies, enabling the agent to balance exploration depth, termination decisions, and recovery from incorrect jumps. In contrast, supervised fine-tuning would require trajectory-level annotations or strong heuristics to approximate long-term returns, which are unavailable in this setting.

### D.3. Language-agnostic Implementation

RepoNavigator is designed to be language-agnostic, relying only on the availability of execution-level navigation primitives provided by a language server, such as symbol resolution or call-site traversal. These primitives are supported across a wide range of programming languages through standard language server protocols. In our experiments, we focus on Python repositories to simplify implementation and evaluation. This choice is made for experimental convenience rather than methodological necessity. The underlying formulation and learning framework do not depend on Python-specific semantics, and can be directly instantiated for other languages given corresponding language server support.

### D.4. Action Space Reduction via a Single Tool

Restricting the agent to a single execution-aware navigation primitive substantially reduces the branching factor of the decision process. For long-horizon tasks with sparse terminal rewards, the probability of success of a trajectory scales multiplicatively with per-step correctness. Expanding the action space with multiple tools increases action entropy and degrades this multiplicative success rate unless strong priors are available. Our empirical results suggest that, in repository-level reasoning, depth of execution-aware exploration is more critical than breadth of tool diversity.

### D.5. Preventing Data Leakage

A common concern is whether pre-training data leakage could undermine the validity of post-training methods. We mitigate this concern through both dataset curation and temporal separation. Specifically, the SWE-bench\_Pro dataset was released in 2025, whereas the Qwen2.5 model family was published in 2024, making direct pre-training exposure unlikely. In addition, we explicitly remove any training samples whose repositories overlap with those in SWE-bench\_Verified or SWE-bench\_Pro. Together, these measures substantially reduce the risk that the observed performance gains arise from data leakage rather than the proposed training procedure.

## E. More Related Works

To enable efficient training, some methods have proposed a simulated environment for agents to interact with. SWE-Gym (Pan et al., 2024) proposed the first Gym environment to train SWE agents. SWE-mirror (Wang et al., 2025a) mirrors issues across datasets to scale up. To stabilize the performance, we use the large-scale training datasets of SWE-smith, which is conservative but has stable performance.

Aside from baseline methods we reproduced in our research, there are also agentic designs for repo-level tasks. PAGENT (Xue et al., 2025) analyzes the failure reasons of existing agents and applies CFG creation and exploration to infer the type of a patch. MarsCode (Liu et al., 2024b) is a training-free multi-agent framework including a manager, a searcher, a tester, a reproducer, and editors. Trae (Gao et al., 2025) is a training-free workflow composed of patch generation, regression test, patch pruning, and selection. Among all agentic frameworks, Trae has the highest rank on the SWE-bench leaderboard. Linxi (Yang et al., 2025c) is a training-free multi-agent framework, while adding the repository as a knowledge database, and retrieval-based methods are applied to extract knowledge from this database. PatchPilot (Li et al., 2025) adds a reproduction and refinement process in the workflow. CORTEXA (Sohrabizadeh et al.) reflects the repository into an embedding space to enable efficient retrieval. Chen et al. (2025c) builds the repository as a graph and applies graph-level searching tools for localization, and Wang et al. (2025b) furthermore integrates commit history as agent memory. RepoLens (Wang et al., 2025c) equips conceptual information of the repository to enable repo-level understanding. These pipelines are training-free and yield competitive results. These workflows are sophisticated, and they successfully integrate repo-level tools, including a language server, terminal, and search engine. However, their complexity hinders their potential for training. On the other hand, we remove unnecessary tools and workflows, preserving only the `jump` tool (built upon the language server) to build a fully-automatic agent. Our design enables efficient training with reinforcement learning.

## F. Illustration of Training

We present the detailed pseudo code of the RL training process of RepoNavigator in Algorithm 1. For simplicity, we include the entry point (normally a testing function) in issue  $q$ .

---

**Algorithm 1** Reinforcement learning process of RepoNavigator using GRPO

---

**Require:** Repository  $\mathcal{R}$ , issue  $q$ , initial policy  $\pi_\theta$ , group size  $G$

**Ensure:** Output  $\hat{Y}$  maximizing  $\mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)]$

```

1:  $o_0 \leftarrow q, \tau \leftarrow \emptyset$ 
2: while not terminated do
3:    $h_t \leftarrow (q, o_{1:t-1}, a_{1:t-1})$ 
4:   Optionally generate intermediate reasoning
5:    $x_t \sim \pi_\theta(\cdot \mid h_t)$ 
6:   if  $x_t$  is a tool-call then
7:      $a_t \leftarrow x_t$ 
8:      $o_t \leftarrow T(a_t, \mathcal{R})$ 
9:      $\tau \leftarrow \tau \cup \{(h_t, a_t)\}$ 
10:  else
11:     $\hat{Y} \leftarrow x_t$ 
12:    break
13:  end if
14: end while
15: Sample a group of trajectories  $\{\tau^{(g)}, \hat{Y}^{(g)}\}_{g=1}^G$  using  $\pi_\theta$ 
16: Compute rewards  $\{R^{(g)} = R(\hat{Y}^{(g)}, Y^*, \tau^{(g)})\}_{g=1}^G$ 
17: Compute group baseline  $\bar{R} \leftarrow \frac{1}{G} \sum_{g=1}^G R^{(g)}$ 
18: for each trajectory  $\tau^{(g)}$  do
19:    $\hat{A}^{(g)} \leftarrow R^{(g)} - \bar{R}$ 
20:   for each  $(h_t, a_t) \in \tau^{(g)}$  do
21:     Assign advantage  $\hat{A}_t \leftarrow \hat{A}^{(g)}$ 
22:   end for
23: end for
24: Estimate GRPO loss  $\mathcal{L}^{\text{GRPO}}(\theta)$  using Eq. 4

```

---

## G. Threats to Validity

**Groundtruth Retrieval** A limitation of our work lies in the extraction of groundtruth locations. We extract modified locations directly from the `gold_patch` in the datasets, which may ignore other patches that also resolve the issue. Our evaluation metrics do not consider these correct alternatives. However, using golden patches is acceptable when comparing multiple methods. If a method reveals golden locations (locations in golden patches), it undoubtedly contributes to the resolution of the issue, and the result in Table 3 demonstrates this claim.

**Language Limit** Another limitation is that we only evaluate Python repositories in our experiments. This is because each language (C/C++, Java, etc.) has its unique language server, and we only succeeded in implementing the language server of Python. We will implement more language servers and validate our approach on more programming languages in the future.

## H. Case Study

In this section, we present the full trajectory of RepoNavigator on *astropy\_astropy-12907* from SWE-bench\_Verified in Fig. 7. We apply the default tool-calling prompt template of verl (Shen, 2024) and present an example. The system prompt and user prompt are presented in Fig. 7. Note that we do not present any process restrictions in our prompt, encouraging RepoNavigator to plan, call tools, and make decisions fully automatically. This is distinct from Agentless (which has a fixed workflow), LocAgent (which predefines a specific step-by-step workflow in its system prompt), CoSIL, and RepoSearcher (which are half-automatic because some forced steps are added to the workflow besides the automatic multi-turn tool-calling conversations).

Figure 7: The following pages are the prompt and a case study on *astropy\_astropy-12907* of SWE-bench\_Verified.

### Prompt

```
[system]
You are Qwen, created by Alibaba Cloud. You are a helpful assistant.

# Tools

You may call one or more functions to assist with the user query.

You are provided with function signatures within <tools></tools> XML tags:
<tools>
{"type": "function", "function": {"name": "jump", "description": "In the
specific file path, a symbol is referred and this tool can find
where the tool is defined.
For instance, in the first turn, file_path is the entry point of.",
"parameters": {"properties": {"symbol": {"description": "The symbol whose
definition code will be given to the agent.", "type": "string"}, "file_path":
{"description": "The relevant path to the file where the symbol is referred.",
"type": "string"}}, "required": ["symbol", "file_path"], "type": "object"}}}
</tools>

For each function call, return a json object with function name and arguments
within <tool_call></tool_call> XML tags:
<tool_call>
{"name": <function-name>, "arguments": <args-json-object>}
</tool_call>
[user]
You are given a codebase and an issue, you need to locate the files and
functions causing this issue.
You can call the tool to check the definition code of a symbol. You can only
check the symbol once for each turn.
The "file_path" is the relevant path of where the symbol is called,
NOT where it is defined!
For instance, if "classA.functionB" is what you want to check (which is called
in fileA.py), you should directly check "functionB" in "fileA.py".

This is the issue:
[Problem Statement]

The entry file of the code base is:
[Relevant Path To Entry Point]
[Entry Point]

Your final answer should be all functions that should be modified, such as:
relevant/path/to/file1.py::func_name1, relevant/path/to/file2.py::func_name2,
...(a series of file::function pairs separated by comma)
Please put your final answer inside \boxed{} only in the last turn.
You can only call the tool once each turn.
For instance:
{"name": "jump", "arguments": {"symbol": "symbol_to_be_checked", "file_path":
"file_where_the_symbol_is_used"}}
```

### User-turn-0

...(system prompt and problem statement are omitted)...

The entry file of the code base is:  
astropy/modeling/tests/test\_separable.py:

```
def test_separable(compound_model, result):  
    assert_allclose(is_separable(compound_model), result[0])  
    assert_allclose(separability_matrix(compound_model), result[1])
```

Your final answer should be all functions that should be modified,  
with the format of:

relevant/path/to/file1.py::func\_name1, relevant/path/to/file2.py::func\_name2, ...  
(a series of file::function separated by comma)

Please put your final answer inside \boxed{} only in the last turn.

### LLM-turn-1

```
<tool_call>  
{  
  "name": "jump",  
  "arguments": {  
    "symbol": "separability_matrix",  
    "file_path":  
      "astropy/modeling/tests/test_separable.py"  
  }  
}  
</tool_call>
```

### User-turn-2

```
<tool_response>  
The definition of symbol "separability_matrix" is:  
def separability_matrix(transform):  
    if transform.n_inputs == 1 and transform.n_outputs > 1:  
        return np.ones((transform.n_outputs, transform.n_inputs), dtype=np.bool)  
    separable_matrix = _separable(transform)  
    separable_matrix = np.where(separable_matrix != 0, True, False)  
    return separable_matrix
```

It is defined in: astropy/modeling/separable.py

You can check other symbols in previous files, or you can give the final answer.  
</tool\_response>

### LLM-turn-3

The "separability\_matrix" function is defined in "astropy/modeling/separable.py". Given the issue with nested CompoundModels, it seems the problem might lie in how the function handles nested models. Specifically, the function might not correctly propagate the separability information through nested models.

To address this, we need to check the "\_separable" function, which is likely where the separability logic is implemented. Let's check the "\_separable" function.

```
<tool_call>
{"name": "jump", "arguments": {"symbol": "_separable", "file_path":
"astropy/modeling/separable.py"}}
</tool_call>
```

**User-turn-4**

<tool\_response>

The definition of symbol "\_separable" is:

```
def _separable(transform):  
    """  
        Calculate the separability of outputs.  
        Parameters  
        -----  
        transform : "astropy.modeling.Model"  
        A transform (usually a compound model).  
        Returns :  
        is_separable : ndarray of dtype np.bool  
        An array of shape (transform.n_outputs,) of boolean type  
        Each element represents the separability of the corresponding output.  
    """  
    if (transform_matrix := transform._calculate_separability_matrix())  
        is not NotImplemented:  
        return transform_matrix  
    elif isinstance(transform, CompoundModel):  
        sepleft = _separable(transform.left)  
        sepright = _separable(transform.right)  
        return _operators[transform.op](sepleft, sepright)  
    elif isinstance(transform, Model):  
        return _coord_matrix(transform, "left", transform.n_outputs)
```

It is defined in: astropy/modeling/separable.py

You can check other symbols in previous files, or you can give the final answer.

</tool\_response>

### LLM-turn-5

The "\_separable" function is defined in "astropy/modeling/separable.py". This function is responsible for calculating the separability of outputs for a given model. Given the issue with nested CompoundModels, it seems the problem might lie in how the function handles the combination of separability matrices for nested models.

To address this, we need to ensure that the separability information is correctly propagated through nested models. Specifically, we need to check how "\_operators" dictionary handles the combination of separability matrices.

Let's check the "\_operators" dictionary.

```
<tool_call>
{"name": "jump", "arguments": {"symbol": "_operators", "file_path":
"astropy/modeling/separable.py"}}
</tool_call>
```

### User-turn-6

```
<tool_response>
The definition of symbol "_operators" is:
_operators = {"&": _cstack, "|": _cdot, "+": _arith_oper, "-": _arith_oper,
"*": _arith_oper, "/": _arith_oper, "**": _arith_oper}
```

It is defined in: astropy/modeling/separable.py

You can check other symbols in previous files, or you can give the final answer.

```
</tool_response>
```

### LLM-turn-7

The "\_operators" dictionary is defined in "astropy/modeling/separable.py". This dictionary maps operators to functions that handle the combination of separability matrices. Given the issue with nested CompoundModels, it seems the problem might lie in how the "\_cstack" function handles the combination of separability matrices.

To address this, we need to check the "\_cstack" function.

Let's check the "\_cstack" function.

```
<tool_call>
{"name": "jump", "arguments": {"symbol": "_cstack", "file_path":
"astropy/modeling/separable.py"}}
</tool_call>
```

### User-turn-8

```
<tool_response>
```

The definition of symbol "\_cstack" is:

```
def _cstack(left, right):
    noutp = _compute_n_outputs(left, right)

    if isinstance(left, Model):
        cleft = _coord_matrix(left, "left", noutp)
    else:
        cleft = np.zeros((noutp, left.shape[1]))
        cleft[: left.shape[0], : left.shape[1]] = left
    if isinstance(right, Model):
        cright = _coord_matrix(right, "right", noutp)
    else:
        cright = np.zeros((noutp, right.shape[1]))
        cright[-right.shape[0]:, -right.shape[1]:] = 1

    return np.hstack([cleft, cright])
```

It is defined in: astropy/modeling/separable.py

You can check other symbols in previous files, or you can give the final answer.

```
</tool_response>
```

### LLM-turn-9

The `"_cstack"` function is defined in `"astropy/modeling/separable.py"`. This function is responsible for handling the combination of separability matrices for the `"&"` operator. Given the issue with nested `CompoundModels`, it seems the problem might lie in how it handles the combination of separability matrices for nested models.\n\nTo address this, we need to ensure that the separability information is correctly propagated through nested models. Specifically, we need to check how the `"_coord_matrix"` function handles the combination of separability matrices.

Let's check the `"_coord_matrix"` function.

```
<tool_call>
{"name": "jump", "arguments": {"symbol": "_coord_matrix", "file_path":
"astropy/modeling/separable.py"}}
</tool_call>
```

User-turn-10

<tool\_response>

The definition of symbol "\_coord\_matrix" is:

```
def _coord_matrix(model, pos, noutp):
    if isinstance(model, Mapping):
        axes = []
    for i in model.mapping:
        axis = np.zeros((model.n_inputs,))
        axis[i] = 1
        axes.append(axis)
    m = np.vstack(axes)
    mat = np.zeros((noutp, model.n_inputs))
    if pos == "left":
        mat[: model.n_outputs, :model.n_inputs] = m
    else:
        mat[-model.n_outputs:, -model.n_inputs:] = m
        return mat
    if not model.separable:
        # this does not work for more than 2 coordinates
        mat = np.zeros((noutp, model.n_inputs))
    if pos == "left":
        mat[:model.n_outputs, : model.n_inputs] = 1
    else:
        mat[-model.n_outputs:, -model.n_inputs:] = 1
    else:
        mat = np.zeros((noutp, model.n_inputs))
    for i in range(model.n_inputs):
        mat[i, i] = 1
        if pos == "right":
            mat = np.roll(mat, (noutp - model.n_outputs))
    return mat
```

It is defined in: astropy/modeling/separable.py

You can check other symbols in previous files, or you can give the final answer.

</tool\_response>

### LLM-turn-11

The "\_coord\_matrix" function is defined in "astropy/modeling/separable.py". This function is responsible for creating an array representing the inputs and outputs of a simple model. Given the issue with nested CompoundModels, it seems the problem might lie in how the function handles the combination of separability matrices for nested models.

To address this, we need to ensure that the separability information is correctly propagated through nested models. Specifically, we need to check how the function handles the combination of separability matrices for nested models. Based on the definitions provided, the issue seems to be with the "\_cstack" function, which is responsible for handling the combination of separability matrices for the "&" operator. The function does not correctly handle the combination of separability matrices for nested models. Therefore, the functions that should be modified are:

```
\\boxed{astropy/modeling/separable.py::_cstack}
```

### Reward Calculator

```
Ground Truth: astropy/modeling/separable.py::_cstack
Function Recall: 1.0
Function Precision: 1.0
Function F1: 1.0
Function IoU: 1.0
File Recall: 1.0
File Precision: 1.0
File F1: 1.0
File IoU: 1.0
```